

Backend.js Test Assignment: Distributed Video Surveillance System

Objective

Develop a prototype of a distributed video surveillance system consisting of multiple interacting components.

Estimated completion time: up to 1 week.

A complete implementation of the mandatory requirements is expected.

The Bonus sections are intended to demonstrate additional competencies and are not required to complete the assignment.

Overall Architecture

The system must consist of the following components:

- RTSP camera simulator
- Server A – Video Server
- Server B – Authorization and Management Server
- Web Client
- Inter-server communication channel between Server A and Server B

All components must run on Windows 11 x64.

1. RTSP Camera Simulator

The candidate must independently organize an RTSP video source.

The following options are allowed:

- ffmpeg
- MediaMTX (RTSP Simple Server)
- any other RTSP server
- a custom implementation

The RTSP stream must be available at:

`rtsp://127.0.0.1:1111/camera`

For simplicity, the following ffmpeg launch example may be used:

JavaScript

```
const path = require('path');

const ffmpegPath = path.join(__dirname, 'ffmpeg.exe');

const rtspUrl = "rtsp://127.0.0.1:1111/camera";

const command =

`${ffmpegPath}

-re

-stream_loop -1

-f lavfi

-i testsrc=size=640x480:rate=30

-vf format=yuv420p

-c:v libx264

-profile:v baseline
```

```
-x264-params "bframes=0"  
  
-an  
  
-f rtsp  
  
${rtspUrl}`;
```

This example may be used without modifications.

The test video stream must contain a visually changing element:

- current time, or
- a seconds counter since startup

This is required to evaluate stream freshness and video delivery latency.

The stream must run continuously and recover automatically after temporary failures.

2. Server A – Video Server

Core Functionality

Server A must:

- receive the RTSP stream;
- convert it into a format supported by modern browsers without plugins;
- provide the video stream to clients.

Allowed technologies include:

- HLS
- MSE + fMP4
- WebRTC
- other modern solutions

The selected technology must be documented.

Performance

The implementation should aim to minimize latency between the RTSP source and video playback in the browser.

The chosen streaming technology should balance latency, implementation complexity, and operational stability.

Access Control

Video must be available only to authorized users.

Simply hiding the video stream URL is not sufficient.

Server A must independently verify a user's access rights.

Allowed approaches include:

- JWT
- one-time tokens
- signed URLs
- custom authorization mechanisms

Stream Control

Server A must support the following commands:

- START_VIDEO
- STOP_VIDEO
- GET_STATUS

START_VIDEO

Resumes video delivery to clients.

STOP_VIDEO

Stops video delivery to clients.

At the same time:

- the RTSP source must continue running;
- ffmpeg and related processes must not be restarted.

GET_STATUS

Returns the current state of the video server.

Minimum response format:

JSON

```
{  
  "videoRunning": true  
}
```

The response structure may be extended.

Inter-Server Communication

Server A receives commands from Server B.

REST APIs must not be used for communication between the servers.

Allowed technologies include:

- TCP sockets
- WebSocket
- named pipes
- gRPC
- Redis Pub/Sub
- Redis Streams
- other solutions

The implementation must correctly restore the connection after interruptions.

3. Server B – Authorization and Management Server

Users

User accounts must be stored in the file:

`users.json`

Example structure:

JSON

```
[
  {
    "login": "admin",
    "passwordHash": "...",
    "role": "operator"
  },
  {
    "login": "viewer",
    "passwordHash": "...",
    "role": "viewer"
  }
]
```

Passwords must be stored only as bcrypt hashes.

The `role` field is used only if Bonus Task A (role-based access control) is implemented.

Number of users:

- 3–4

Authentication

POST /auth/login

Request:

JSON

```
{  
  "login": "admin",  
  "password": "secret"  
}
```

Upon successful authentication, the server must issue a JWT.

JWT lifetime:

- 1 hour

A refresh token mechanism is not required.

Authorization Validation

GET /protected

Validates the JWT.

WebSocket

After successful authentication, the client opens a WebSocket connection to Server B.

The JWT must participate in the connection authentication process.

Supported Commands

The client may send the following commands:

- START_VIDEO
- STOP_VIDEO
- GET_STATUS
- LOGOUT

LOGOUT

This command must:

- terminate the user's session;
- close the WebSocket connection;
- revoke access to video;
- require re-authentication before further use.

Forwarding Commands to Server A

Video control commands must be forwarded to Server A through the inter-server communication channel.

REST APIs must not be used for this purpose.

Logging

All commands and control messages received by Server B must be written to:

`commands.log`

Minimum log entry format:

JSON

```
{  
  
  "timestamp": "...",  
  
  "user": "admin",
```

```
"message" : "STOP_VIDEO"  
}
```

Everything received from the client and involved in command processing must be logged.

Additional credit will be given for protection against log tampering, such as:

- HMAC
 - integrity verification
 - append-only logging
 - other approaches
-

4. Web Client

After system startup, a web page must be available.

Login Screen

Must contain:

- login field
- password field
- login button

Main Screen

Video Player

Receives video from Server A.

Management Console

The client must provide a text input field for entering commands.

Example:

None

START_VIDEO

STOP_VIDEO

GET_STATUS

LOGOUT

After sending a command, the client must display the server response.

Example:

None

> GET_STATUS

< VIDEO_RUNNING

> STOP_VIDEO

< OK

> GET_STATUS

< VIDEO_STOPPED

Status Display

The client must display:

- server responses
- command execution errors
- connection errors

Separate control buttons are not required.

5. Documentation

A page must be implemented at:

[/docs](#)

The page must describe:

Architecture

Component interaction diagram.

Technologies Used

List of technologies and rationale for their selection.

Setup Instructions

Step-by-step instructions for running the system.

Architectural Decisions

Description of ambiguities in the specification and the chosen solutions.

Limitations

What was not implemented and why.

6. Runtime Environment Requirements

Operating system:

- Windows 11 x64

Allowed tools and technologies:

- Node.js LTS
- ffmpeg
- ffmpeg-static
- MediaMTX
- Redis
- any open-source libraries

Docker and containerization are not required.

7. Submission Format

A link to a public Git repository must be provided.

The repository must contain:

- README.md
- SOLUTION.md

README

Must contain a brief setup guide.

SOLUTION

Must contain a description of architectural decisions.

Additional credit will be given for:

- architecture diagrams
 - screenshots of the working system
 - demonstration video
-

8. Bonus Tasks

Implementation of bonus tasks is optional.

Bonus A. Role-Based Access Control

Implement user roles.

viewer

Allowed:

- GET_STATUS
- video viewing

operator

Allowed:

- GET_STATUS
- START_VIDEO
- STOP_VIDEO
- video viewing

Bonus B. End-to-End Asymmetric Command Encryption

Implement encrypted command transmission from the client to Server B.

Example:

None

Client

↓

RSA/ECC ciphertext

```
↓  
Server B  
  
↓  
decryption  
  
↓  
command processing
```

The public key must be provided through a dedicated HTTP endpoint.

Server B must decrypt received commands before processing them.

Bonus C. Native C++ Addon for Node.js

Implement a native module using N-API.

Minimum interface:

```
C/C++  
generateKeyPair()  
  
getPublicKey()  
  
decryptWithPrivateKey()
```

All operations must be asynchronous and must not block the Node.js event loop.

Bonus D. TPM Integration

Use TPM for private key storage.

If TPM is unavailable:

- software emulation is allowed;
- solution limitations must be documented.

The private key must never leave the TPM.